От учащегося 1 курса

магистратуры ВШЭ

«Инженерия данных»

Базунова Алексея Витальевича

Итоговое задание по модулю 3

Нереляционные базы данных

Задача проекта - Научиться разрабатывать проект базы данных с горизонтальным масштабированием (шардингом), включая проектирование схемы, реализацию распределённой системы, создание клиентского интерфейса и валидацию производительности под нагрузкой. Продемонстрировать комплексное понимание современных подходов к работе с большими данными.

Этот проект я выполняю на локальной виртуальной машине с Ubuntu, используя Oracle VirtualBox. Помимо этого отчета, в папке проекта находится файл README, в котором можно найти инструкции по запуску. В ходе этой работы я возьму БД из прошлого итогового задания (отчёт и подробная архитектура к БД также находится в этом репозитории в папке /материалы). Кратко напомню самое важное.

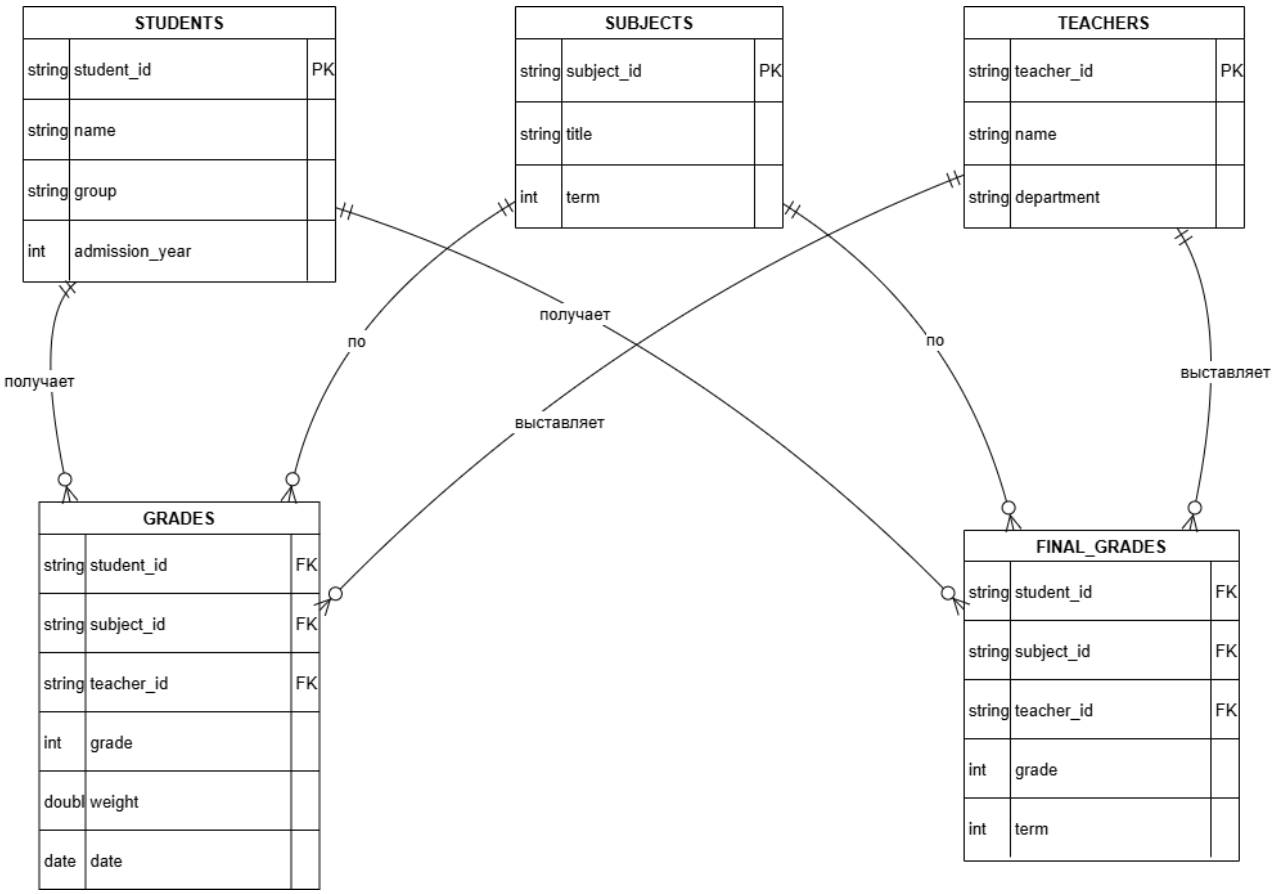
В БД следующие коллекции:

- students (студенты)
- teachers (преподователи)
- subjects (учебные предметы)
- grades (оценки (с весами)
- final_grades (итоговые оценки (за семестр))

Индексы:

- Уникальные первичные индексы:
- student_id, subject_id, teacher_id
-

Схема БД:



Реализуем CLI:

За клиентский интерфейс отвечает программа «main.py» в папке nosql_cli. Выглядит он так:

```
(venv) alex@alex-VirtualBox:~/nosql_cli$ python main.py

1 - Показать оценки студента
2 - Добавить оценку
3 - Средний балл студента
4 - Итоговые оценки студента
0 - Выход
```

Выполняет, соответственно указанные задачи, например, «Показать оценки студента»:

```
> 1
student_id: S1238
{'student_id': 'S1238', 'subject_id': 'NS1', 'teacher_id': 'T136', 'grade':
 7, 'weight': 0.5}
{'student_id': 'S1238', 'subject_id': 'NS1', 'teacher_id': 'T136', 'grade':
 8, 'weight': 0.5}
```

Чтобы не нагружать отчет, другие примеры работы можно найти на скриншотах в папке «материалы».

Реализуем первичное тестирование:

Перед реализацией шардинга проведем тестирование. За оба тестирования отвечает файл «benchmark.py» в папке /nosql_cli. Он производит операции массовой вставки оценок (20000), чтения по «students_id» (1000) и агрегацию через вычисление среднего балла (300). Замеряем результаты:

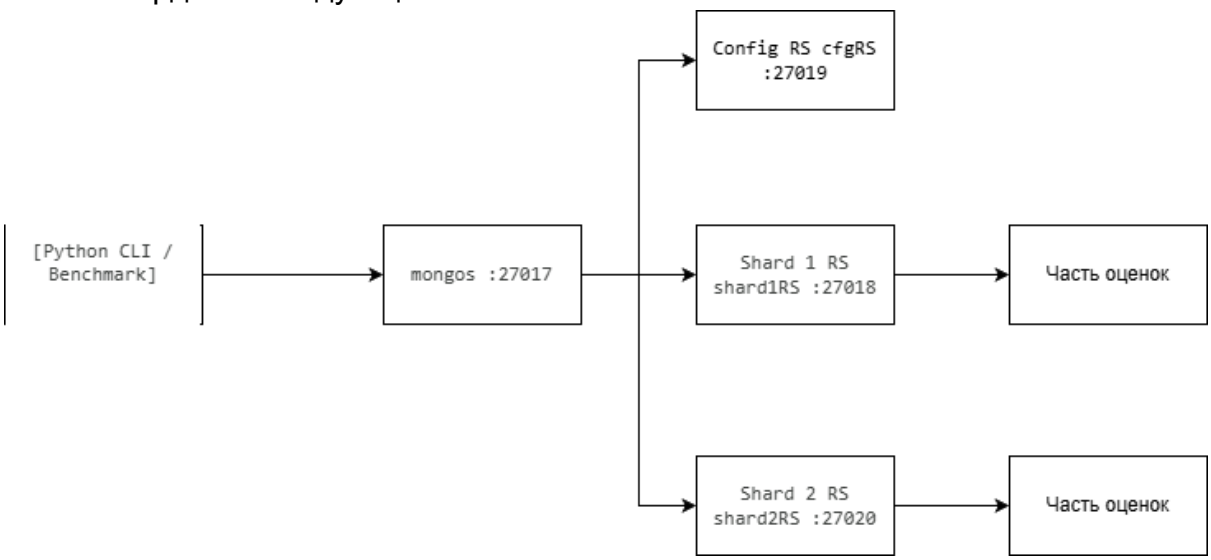
```
(venv) alex@alex-VirtualBox:~/Desktop/project/nosql_cli$ python3 benchmark.
py
INSERT total_docs=20000
total_time=0.2898 sec
avg_batch_time=0.0117 sec
throughput=69011.14 docs/sec
READ iterations=1000
total_time=0.3586 sec
avg_latency=0.3563 ms
throughput=2788.57 ops/sec
AGGREGATE iterations=300
total_time=0.3043 sec
avg_latency=1.0092 ms
throughput=985.85 ops/sec
```

Реализуем шардинг:

Шардинг я реализую через создание кластера на одной машине из нескольких mongodb на разных портах(config, shard1, shard2, mongos).

Поставлю гипотезу: такая архитектура на одной машине с одной ограниченной мощностью скорее всего не окажется «продуктивной», тем более учитывая малое количество данных. Тестирование после шардинга скорее всего покажет общее замедление работы БД. Но, поскольку наша задача - научится работать с шардингом, такой результат тоже приемлем.

Схема шардинга следующая:



Кластер состоит из config server, двух shard replica set, и роутера. Создаем отдельные каталоги, затем запускаем процессы на портах:

```
alex@alex-VirtualBox:~/Desktop/project$ mongod --configsvr --replSet cfgRS
--dbpath ~/mongo-cluster/config --port 27019 --bind_ip 127.0.0.1 --fork --l
ogpath ~/mongo-cluster/logs/config.log
about to fork child process, waiting until server is ready for connections.
forked process: 6369
child process started successfully, parent exiting
alex@alex-VirtualBox:~/Desktop/project$ mongod --shardsvr --replSet shard1R
S --dbpath ~/mongo-cluster/shard1 --port 27018 --bind_ip 127.0.0.1 --fork -
-logpath ~/mongo-cluster/logs/shard1.log
about to fork child process, waiting until server is ready for connections.
forked process: 6459
child process started successfully, parent exiting
alex@alex-VirtualBox:~/Desktop/project$ mongod --shardsvr --replSet shard2R
S --dbpath ~/mongo-cluster/shard2 --port 27020 --bind_ip 127.0.0.1 --fork -
-logpath ~/mongo-cluster/logs/shard2.log
about to fork child process, waiting until server is ready for connections.
forked process: 6535
child process started successfully, parent exiting
alex@alex-VirtualBox:~/Desktop/project$ mongos --configdb cfgRS/127.0.0.1:2
7019 --bind_ip 127.0.0.1 --port 27017 --fork --logpath ~/mongo-cluster/logs
/mongos.log
{"t":{"$date":"2026-03-18T08:58:55.924Z"},"s":"W",  "c":"SHARDING", "id":24
132,   "ctx":"main","msg":"Running a sharded cluster with fewer than 3 conf
ig servers should only be done for testing purposes and is not recommended
for production."}
about to fork child process, waiting until server is ready for connections.
forked process: 6611
```

Подключаем шарды.

```
[direct: mongos] test> sh.addShard("shard1RS/127.0.0.1:27018")
| sh.addShard("shard2RS/127.0.0.1:27020")
| sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('69ba6c2a953a2cf2c64940ea') }
---
shards
[
  {
    _id: 'shard1RS',
    host: 'shard1RS/127.0.0.1:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1773825187, i: 11 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shard2RS',
    host: 'shard2RS/127.0.0.1:27020',
    state: 1,
    topologyTime: Timestamp({ t: 1773825187, i: 29 }),
    replSetConfigVersion: Long('-1')
  }
]
---
active mongoses
[ { '8.0.20': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
```

Загружаем данные. Ключем для коллекции оценок «grades» будет «student_id». Используем hashed стратегию. Проверяем работу шардинга командой «db.grades.getShardDistribution():

```
[direct: mongos] nosql_hw> db.grades.getShardDistribution()
Shard shard1RS at shard1RS/127.0.0.1:27018
{
  data: '1008B',
  docs: 9,
  chunks: 1,
  'estimated data per chunk': '1008B',
  'estimated docs per chunk': 9
}
---
Shard shard2RS at shard2RS/127.0.0.1:27020
{
  data: '224B',
  docs: 2,
  chunks: 1,
  'estimated data per chunk': '224B',
  'estimated docs per chunk': 2
}
---
Totals
{
  data: '1KiB',
  docs: 11,
  chunks: 2,
  'Shard shard1RS': [
    '81.81 % data',
    '81.81 % docs in cluster',
    '112B avg obj size on shard'
  ],
  'Shard shard2RS': [
    '18.18 % data',
    '18.18 % docs in cluster',
    '112B avg obj size on shard'
  ]
}
```

)»:

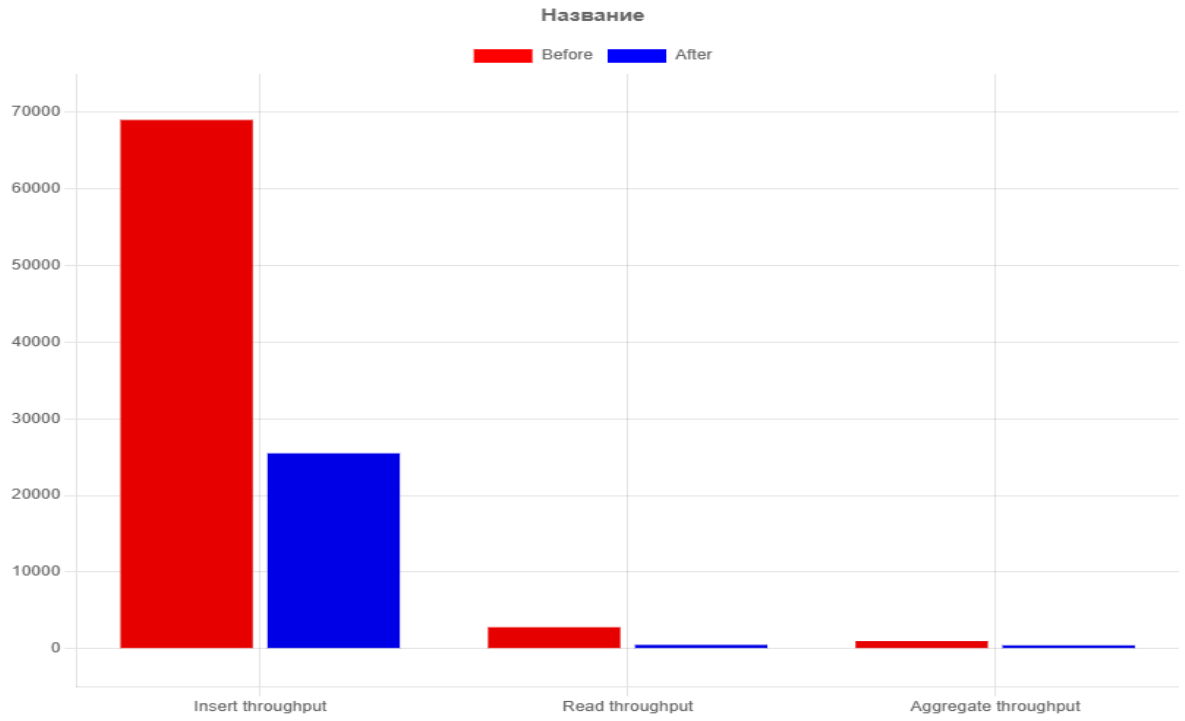
Видим успешную реализацию шрадинга, наши оценки разбились по двум шардам (2 и 9 документов). Конечно, такое соотношение не самое оптимальное, но шардинг реализован.

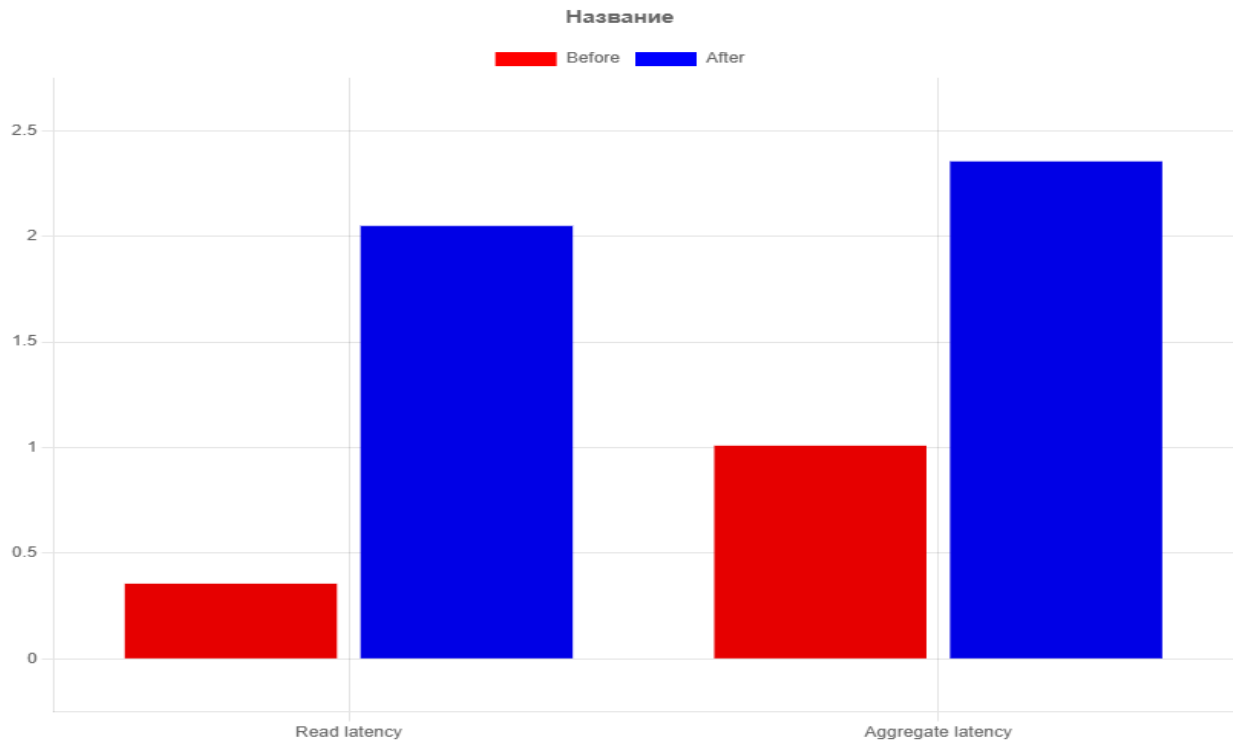
Реализуем конечное тестирование:

Замеряем результаты уже после шардинга:

```
(venv) alex@alex-VirtualBox:~/Desktop/project/nosql_cli$ python benchmark.p
y
INSERT total_docs=20000
total_time=0.7847 sec
avg_batch_time=0.0333 sec
throughput=25488.94 docs/sec
READ iterations=1000
total_time=2.0536 sec
avg_latency=2.0492 ms
throughput=486.94 ops/sec
AGGREGATE iterations=300
total_time=0.7082 sec
avg_latency=2.3542 ms
throughput=423.58 ops/sec
```

Сравнение в виде гистограмм:





Видим, как и предполагалось, что скорость работы шардированного кластера значительно упала по сравнению с одиночным mongodb. Я полагаю, это произошло из-за затрат на координацию между шардами, маршрутизацию, работу mongos и т.п..

Выводы

Таким образом, задача успешно выполнена: реализован консольный интерфейс, шардинг коллекции «grades», а также проведено тестирование до/после с подтверждением гипотезы.